

Computer Organization Project 4 – Instruction Level Parallelism

Due: 23:59, June 3, 2026

A superscalar processor is capable of issuing and executing multiple instructions per clock cycle by leveraging instruction-level parallelism (ILP). This is achieved through wider datapaths, multiple execution units, and dynamic scheduling mechanisms, such as Tomasulo's algorithm. Key features include register renaming, reservation stations, in-order issue, out-of-order execution, and in-order retirement. LLVM-MCA (LLVM Machine Code Analyzer) is a static analysis tool that models the performance of machine instructions on a given target CPU. Rather than executing code on either real hardware or a microarchitectural simulator (e.g., gem5), LLVM-MCA analyzes the scheduling and backend behavior of instructions directly from assembly code generated by a compiler. It provides detailed insights through timeline visualizations and throughput metrics, making it especially suitable for analyzing ILP and identifying potential bottlenecks in superscalar processor designs.

In this project, you will use LLVM-MCA to explore the behavior of superscalar processors through static performance modeling. You will analyze how instruction-level parallelism (ILP) is affected by both program structure and configurable microarchitectural parameters, such as issue width and the availability of functional units. LLVM-MCA simulates instruction scheduling on a target CPU and provides cycle-by-cycle timeline views that help you identify pipeline utilization and performance bottlenecks. You are encouraged to experiment with the following microarchitectural parameters by modifying the target description file used by LLVM-MCA: RISCVSchedTTAscalonX.td.

- IssueWidth: Number of instructions that can be issued per cycle
- Reorder Buffers: Size of internal buffers (optional)
- Functional Unit Counts: e.g., FXA, FXB, FP, Load/Store units

[Hint] More parameters can be found via the configuration file. URL:

<https://github.com/llvm/llvm-project/blob/main/llvm/lib/Target/RISCV/RISCVSchedTTAscalonX.td>

In addition, you are encouraged to use LLVM-MCA to analyze the scheduling behavior of your test programs under various microarchitectural configurations. Identify and explain some of the following behaviors using timeline views and execution statistics. Each observation should be accompanied by a screenshot with instruction labels and your explanation:

- In-order issue
- Out-of-order execution
- In-order commit (retirement)

- Multiple instructions issued or committed in the same cycle
- Pipeline bubbles caused by data hazards, control hazards, or resource stalls, etc.
- [BONUS] Evidence of register renaming eliminating WAW or WAR hazards.
- [BONUS] Any additional phenomena you discovered through LLVM-MCA analysis.

Please submit your research and experiment report along with your C/C++ test programs and microarchitectural parameter configurations according to the following rules:

- 1- You need to install the GNU RISC-V 64-bit cross-compiler GCC/G++ toolchain and compile your test programs with -static and -O0 compilation options.
- 2- Design at least two test programs: one with high ILP (e.g., low data dependencies) and the other with low ILP (e.g., serial data dependencies). Simulate each program under at least two microarchitectural configurations. Compare their performance and pipeline behaviors. Explain how instruction dependency interacts with microarchitecture design and what kinds of programs benefit most from superscalar processors.
- 3- Write your research and experiment report to discuss your evaluation results and summarize your observations.

Reference:

[1] The LLVM project

URL: <https://github.com/llvm/llvm-project>

[2] LLVM-MCA Guide

URL: <https://llvm.org/docs/CommandGuide/llvm-mca.html>

[3] Tenstorrent TT-Ascalon RISC-V CPU IP

URL: <https://tenstorrent.com/ip/risc-v-cpu>